ASSIGNMENT 1

PAYMENT CALCULATOR FOR A VILLAGE MILK CO-OPERATIVE

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

1. AIM

To evaluate the milk payment calculator and recommend the most suitable design based on accuracy, readability, reusability, and maintainability.

2. OBJECTIVE

To analyse the different design approaches and justify why using functions, constants, meaningful variable names, and the Decimal data type is suitable for the milk payment calculator.

3. DESIGN APPROACH

The milk payment calculator can be designed using the following approaches.

Approach 1: Writing all calculations in a single block of code.

This approach is simple for small programs, but it becomes difficult to maintain and reuse when the program grows.

Approach 2: Using functions and constants.

This approach separates the calculation logic into functions and stores fixed pricing values as constants. It improves readability, reusability, and maintenance.

Approach 3: Using separate modules.

This approach divides the program into multiple Python files. It is useful for larger applications, but it may introduce unnecessary complexity for a small calculator.

4. EVALUATION OF THE FINAL DESIGN

Accuracy: The Decimal data type helps maintain accurate financial calculations.

Readability: Meaningful variable names and functions make the program easier to understand.

Reusability: A function such as calculate_payment() can be called for different farmers.

Maintainability: Constants allow pricing rules to be updated easily without changing the entire program.

Testing: Separate functions make it easier to test the calculation logic with different inputs.

5. RECOMMENDED DESIGN

The recommended design is to use Python functions, named constants, meaningful variables, and the Decimal data type.

The calculate_payment() function calculates the milk rate per litre and the total payment. Constants store the base rate, base fat percentage, base SNF percentage, and adjustment values.

This design is suitable because it is accurate, readable, reusable, and easy to test. A separate module can be used when the application becomes larger.

